

School Management Platform

Technical Proposal & Software Design Document

AI Fitrah Pre School, Sarjapura — Sompura Gate, Sarjapura, Bengaluru

Document	Technical Proposal & Software Design Document (SDD)
Version	1.0
Date	7 August 2026
Prepared for	AI Fitrah Pre School, Sarjapura
Prepared by	DVLN Solutions LLP
Reference	School Management Platform Brief, August 2026
Status	Draft for client review
Classification	Confidential — commercial in confidence

Revision history

Version	Date	Author	Change
1.0	07-Aug-2026	DVLN Solutions LLP	Initial issue

Table of Contents

1. Executive Summary
2. Scope & Objectives
3. Architecture
4. Data Model
5. Security & Compliance
6. Cost Model
7. Module Designs
8. Delivery Sequence
9. Commercials
10. Risks & Mitigations
11. Assumptions, Dependencies & Acceptance
 - Appendix A — Environment & Secrets
 - Appendix B — Operational Runbook Status
 - Appendix C — Glossary

1. Executive Summary

This document proposes the completion of a School Management Platform for Al Fitrah Pre School, Sarjapura, and records the technical design of the system that already runs in production.

Where the work stands. A public marketing website, an admissions CRM, a student register, a daily attendance register, an immutable fee ledger, a news & events CMS, a read-only parent portal and a permission-aware resource centre are built, deployed and operating. Measured against the August 2026 brief, this represents roughly one third of the requested platform.

What remains. Staff and HR records, academic records and report cards, timetabling, role-specific dashboards, deeper analytics, two-way parent communication over WhatsApp/SMS, and online fee collection. These are described module by module in Section 7 and sequenced in Section 8.

The engineering position. Three commitments shape every design decision in this document and are the reason the platform can be extended rather than rewritten:

1. **Configuration is data, not code.** Pipelines, programmes, class sections, fee methods, attendance statuses and school identity are edited in the admin panel and take effect without a deployment.
2. **The running cost today is ₹0, against a ₹1,000/month budget.** The system is designed to stay inside Google's free tier, leaving the stated budget entirely as headroom. Section 6 shows the arithmetic — and Section 6.5 is honest that this is a starting position: the intended direction, once funding allows, is a move onto Cloudflare and AWS for control and portability, at a real monthly bill.
3. **No browser is trusted with school data.** Every record of consequence — leads, students, attendance, payments, audit entries — is denied to all client access at the database level and reachable only through authenticated server code.

What is asked of the school. Three items in this document require the school's answer rather than engineering effort: a named grievance officer (a legal requirement under the DPDP Act), confirmation of the module priority order in Section 8, and sign-off on the commercial model in Section 9.

2. Scope & Objectives

2.1 Purpose

To deliver a single system covering the school's public presence, admissions, enrolment, daily operations and parent communication, replacing the current mix of spreadsheets, WhatsApp groups and manual record-keeping — with a running cost that does not scale with the number of families.

2.2 Honest position against the brief

Nothing in this section is a plan presented as progress. Current state, module by module:

Module	State	Gap
Public website	Built	No gallery; academics split across Programs/Syllabus
Admissions CRM	Built	—
Admissions pipeline	Built , configurable	—
Teacher recruitment	Built (applications, CV, pipeline)	Interview scheduling, ratings, internal comments
Student records	Built	—
Attendance	Built (student, daily, monthly %)	Staff attendance, bulk marking, export, analytics
Fees	Built (immutable ledger, derived balance)	Online collection, invoices, reminders
Website CMS	Built (news & events)	Gallery, faculty, per-page SEO
Parent portal	Built (read-only)	Two-way messaging, downloads by child
Resource centre	Built	Student & teacher audiences pending identity
Notifications	Built (engine + email + in-app)	WhatsApp and SMS channels unconfigured
Audit log	Built (append-only)	—

Module	State	Gap
Configuration	Built (admin-editable)	New module sections as modules land
Staff / HR	Not started	Whole module
Academics (marks, report cards)	Not started	Whole module
Timetable	Not started	Whole module
Dashboards	One (Insights)	Principal, Admissions, CRM, Attendance, HR, System
Analytics	Partial (sources, funnel, referrers)	Trends, campaign performance, monthly reports
Library / Transport	Not started	Whole modules — optional

2.3 Objectives

Functional

- F1 — One record of every family from first enquiry to enrolment, with no re-keying.
- F2 — Daily operations (register, fees, notices) performable by non-technical staff.
- F3 — Content and configuration changeable by the school without a developer.
- F4 — Parents able to see their own child's record, and nobody else's.
- F5 — Every consequential action attributable to a named person, after the fact.

Non-functional

Ref	Requirement	Target
N1	Running infrastructure cost	₹0/month steady state; ₹1,000/month ceiling
N2	Public page load	Static or incrementally regenerated; zero database reads per visit
N3	Admin page cost	Constant in record count — 500 or 50,000 students cost the same per page
N4	Data residency	Children's data stored in India
N5	Recovery	Point-in-time recovery enabled; weekly off-site export
N6	Availability	Managed platform; no self-hosted server to patch or restart

Ref	Requirement	Target
N7	Compliance	DPDP Act 2023, including the child-data provisions

2.4 Out of scope

The following are deliberately excluded. Each is excluded for a stated reason, not by oversight, and each can be added later as a separate engagement.

Excluded	Reason
Full-text search across leads	Requires a paid search service (Algolia/Typesense) — breaks the cost target. Filtered and paginated search is provided instead.
Rich-text (WYSIWYG) editor for posts	Treating editor output as markup makes the CMS a stored-XSS surface. Post bodies are plain text rendered as paragraphs. A sanitising editor is a project, not a component swap.
Real-time listeners in the admin console	Each open browser tab bills database reads continuously. The console uses optimistic updates instead.
Charts in the current Insights view	A charting bundle is page weight for no additional decision value at present volumes. Revisited with the dashboards module.
Native mobile applications	The platform is a responsive web application, installable to the home screen.
Biometric / RFID attendance hardware	Hardware procurement and on-site integration; separate engagement.
Accounting-system integration (Tally/Zoho)	No requirement stated in the brief; the fee ledger exports to CSV.

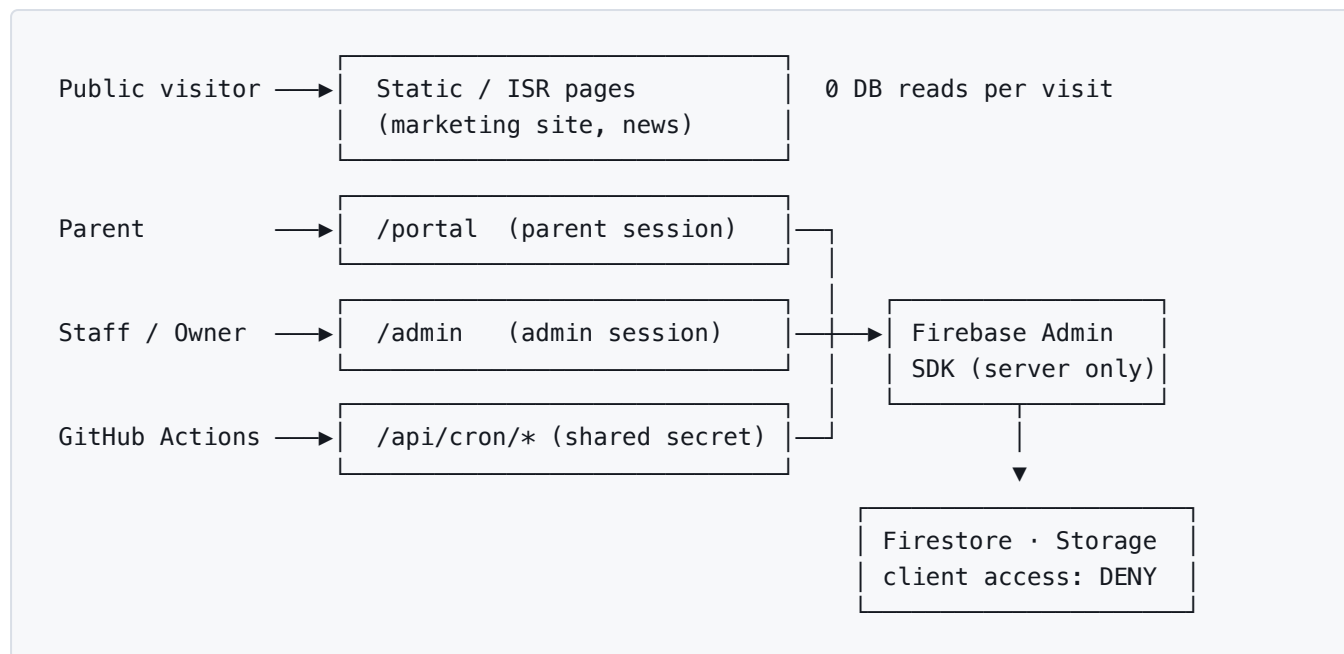
3. Architecture

3.1 Technology stack

Layer	Technology	Why
Application	Next.js 16 (App Router), React 19, TypeScript 5	One codebase serves the marketing site, the admin console and the parent portal. Server components keep database credentials off the browser entirely.
Styling	Tailwind CSS 4	No component library to version-chase; the design system is the stylesheet.
Validation	Zod 4	One schema validates the form, the API route and the stored document — a single definition, not three that drift.
Database	Cloud Firestore (asia-south1 , Mumbai)	Managed, serverless, generous permanent free tier, and no server to patch.
Files	Cloud Storage	Same trust domain; lifecycle rules bound growth automatically.
Identity	Firebase Authentication	Google sign-in for staff; email-link and phone sign-in for parents. 50,000 monthly active users free.
Hosting	Firebase App Hosting (Cloud Run, asia-east1)	Deploys from Git; scales to zero; no fixed monthly instance cost.
Email	Resend	Transactional email with a free tier adequate for present volume.
Testing	Vitest (unit), Playwright (end-to-end)	Both run in CI on every change.
CI/CD & scheduling	GitHub Actions	Build, lint and test on every pull request; also drives the daily follow-up digest and the weekly database export — no separate scheduler service to pay for or monitor.

No new runtime dependency is added without an explicit call-out and written sign-off. The current production dependency list is ten packages.

3.2 Runtime topology



Every path to data passes through server code that has already established who the caller is. There is no second path.

3.3 The central design decision — configuration as data

The brief states: *"NEVER hardcode mutable values. Everything configurable. Everything editable via Admin Panel."*

Ten values previously lived as TypeScript constants: admission and recruitment pipelines, stage metadata, class sections, programmes, lead sources, employment types, post types, attendance statuses, student statuses and payment methods — plus the school's own identity.

Every module in the brief consumes at least one of them. **Building the remaining modules first and making configuration editable second would mean rewriting all of them.** So configuration became data before the modules land on top of it.

Design. One `settings/platform` document, wrapped in a request-and-process cache keyed by a tag and invalidated on write. Steady state is *one* database read per revalidation window shared across every request — not one read per request, which would have added roughly ten reads to every page render and undone the cost work described in Section 6.

Defaults stay in code, and the stored document is deep-merged over them. Three deliberate consequences:

1. Nothing breaks before configuration exists — the platform runs with an empty settings collection.
2. A new setting ships with a working value instead of requiring a data migration.
3. A partial or corrupt settings document degrades to the default rather than taking the site down. A single configuration document is otherwise a single point of failure for the entire site.

Two values deliberately remain in code, and the reasoning is recorded because it will look like an inconsistency otherwise:

- The **root error page's phone number**. That page may well be rendering *because* the database is unreachable. A fallback that needs a working database read is not a fallback.
- The **coming-soon gate**. It is evaluated in an edge runtime on every request that cannot read Firestore. A copy in settings would create two sources of truth that can disagree. The settings screen displays it read-only.

3.4 Rendering and caching strategy

Surface	Strategy	Database reads
Marketing pages	Static at build	0
News, events, careers	Incremental regeneration, revalidated on edit	~5 per hour, shared by all visitors
Admin console	Server-rendered per request, paginated	Constant per page, independent of collection size
Parent portal	Server-rendered per request	Bounded by the number of that parent's children
Configuration	Cached with tag invalidation	~1 per hour, shared

Public visitors never cause a database read. This is what makes a marketing campaign driving thousands of visitors cost nothing.

3.5 Extension seams for ERP

The brief requires that later modules — student profiles, parent portal, marks, timetable, library, transport — attach without refactoring. Three seams carry that, and all three are built:

3.5.1 Configuration registry. A new module declares its own settings section with a schema and defaults. No change to existing settings code, no migration.

3.5.2 Notification engine. Callers emit a domain event — `notify(event, payload)` — and never name a channel. Which channels fire, and what they say, come from configuration. Adding WhatsApp means implementing one adapter's `send` function; no route, action or template changes. WhatsApp and SMS are declared today and report themselves unconfigured, so their toggles are honest about doing nothing yet.

`notify()` never throws. A parent's enquiry must be saved even if every channel is down — losing the lead in order to announce the lead is the worst available trade. Channels dispatch in parallel and are isolated

from one another, so a failing email provider cannot prevent the in-app notice being written. Verified by submitting a real enquiry with an invalid email API key: the lead saved, the dashboard notice delivered.

3.5.3 Audit log. Append-only, written on the *same database batch* as the action it records, so an audited action cannot succeed while its record silently fails.

4. Data Model

4.1 Collections

Collection	Holds	Growth	Client access
leads	Admission enquiries and job applications	Per enquiry	Denied
leads/{id}/notes	Activity timeline — notes and stage changes	Per interaction	Denied
students	Enrolled children, guardians, fee cache	Per admission	Denied
attendance	One document per class per day	~132/month for six classes	Denied
payments	Immutable fee ledger	Per payment	Denied
resources	File metadata and audience	Per upload	Denied
posts	News and events	Per post	Read if published
notifications	In-app notices for staff	Per event	Denied
auditLog	Append-only action record, 2-year TTL	Per action	Denied
settings	One configuration document	Fixed	Denied

4.2 Modelling decisions worth defending

4.2.1 Students are their own collection, not a third kind of lead. `leads` already carries two shapes that disagree (`parentName` vs `name` , `childAge` vs `role`), and every query filters on type first. A third disjoint field set behind the same discriminator means every read fetches fields it cannot use and every index carries documents it cannot match. A student also has a different lifecycle: a lead ends, a student persists for years and accumulates attendance and fees.

A lead does not *become* a student — it *produces* one, and the lead survives as the record of how the family arrived, which the funnel and referral reporting still count.

Admission numbers are allocated inside a transaction, derived from the highest already issued. Two staff enrolling simultaneously would otherwise mint the same number — the exact identifier the school uses to tell two children apart.

4.2.2 Attendance is one document per class-day, not per student-day. For a six-class school of twenty children each, over a 22-day month:

Shape	Writes/month	Reads to open one register
Per class-day (chosen)	132	1
Per student-day (naive)	2,640	20

Twenty times the writes and twenty times the reads, for a strictly worse query. The document identifier is derived — `{academicYear}_{section}_{date}` — so marking is idempotent (re-submitting overwrites rather than duplicating) and opening today's register is a single lookup by identifier, with no query and no index at all.

4.2.3 Fees are an immutable ledger with a derived balance. The obvious design — a mutable "amount paid" field that each payment overwrites — loses money. Two staff recording payments at the same moment both read ₹5,000, both write ₹7,000, and one family's ₹2,000 disappears with no error and no trace. It is the one class of failure in this system that cannot be reconstructed afterwards, because the evidence is the thing that was overwritten.

Payments are therefore immutable rows. The student's paid total is maintained with an atomic server-side increment, so concurrent payments add rather than clobber, and it is treated as a *cache*: a reconcile routine recomputes it from the ledger, and the ledger always wins. A correction is a new negative row — a refund or an adjustment — never an edit.

4.2.4 Unbounded growth lives in subcollections. Lead notes were an array on the lead document. Three problems at once: the 1 MiB document ceiling would permanently brick an active lead, every note rewrote the whole document, and the inbox paid to transfer note history it used only to count. Notes moved to a subcollection with a denormalised count.

4.3 Cost invariants

These are enforced by an automated test (`tests/unit/costInvariants.test.ts`). A change violating one is rejected regardless of what else it improves.

1. No unbounded collection reads — every query is bounded, counted, or a document lookup.
2. Counts come from aggregation queries, never from fetching documents to count them.
3. Public pages never read the database per request — static or incremental only.
4. Unbounded growth lives in subcollections, not documents.
5. No placeholder `null` in an indexed field.

6. Cloud Storage has a lifecycle rule.
7. Configuration is cached, never read per request.

Invariant 5 is not theoretical. Writing `followUpDate: null` as a placeholder caused the daily reminder job to sweep every lead ever created, because Firestore indexes an explicit null and orders it before every timestamp.

5. Security & Compliance

5.1 Trust boundary

Firestore security rules deny **all** client read and write access to `leads`, `students`, `attendance`, `payments`, `resources`, `notifications` and `auditLog`. Published posts are readable because they are, by definition, public. Everything else denies by default.

This is not defence in depth applied to an otherwise open API — it is the primary control. Data is reachable only through server code holding the Admin SDK, which bypasses rules *and* has already established the caller's identity. A vulnerability in the browser bundle cannot reach a single student record, because the browser was never given a route to one.

The recursive wildcard is used deliberately on each protected collection: a rule matching one path segment would not cover subcollections, so the lead notes timeline would be left open by a non-recursive rule even though the parent looked protected.

5.2 Staff authentication and authorisation

Staff sign in with Google. Access requires the address to be present in an allowlist; a second, narrower list confers owner rights. The split exists because a flat list meant the receptionist logging a phone call had identical rights to the owner — every CV, every child's date of birth, and delete on any record.

Roles come from environment configuration rather than authentication token claims. The list is three or four addresses that change once a year; environment variables need no migration, no claim-setting script, and carry no risk of a user whose token predates their role change. This is recorded as an explicit trade with a stated upgrade path: move to token claims if staff ever need to manage roles themselves.

5.3 Parent identity — the rule this system exists to enforce

Parents sign in by **email link** where the school holds an email address, and by **phone code** where it does not.

Email is the default because it is free: every verification SMS is billed with no free allowance, and India sits in an expensive band. Phone is retained as the fallback because the school's own records require it — a phone number is mandatory on the enquiry form, the CSV import and every guardian record, while an email is optional in all three. Email-only sign-in would lock out most existing families.

Which children a signed-in parent may see is resolved on the server, from the verified credential in their session. It is never taken from a URL, a form field, or anything else a browser can set. A parent

seeing another family's fees or medical notes is the worst failure this system can have, and the only reliable defence is that the client is never asked.

5.4 File access — the storage prefix is the permission

Who may see a file is decided by *where the file is stored*, not only by a field on a document:

Audience	Stored under	Served by
Public	<code>content/</code>	Cloud Storage directly — world-readable, cached, free, no request touches the application
Parents, staff	<code>resources/</code>	Streamed through an authenticated route; closed to every client by storage rules

Changing a file's audience across that boundary **moves the object**. Without that, un-publishing a file would leave it at a stable, world-readable URL that anyone who once saw it can still fetch — precisely the leak the rules exist to prevent.

CVs are **streamed through an authenticated route**, never served by a time-limited signed URL. A signed URL is a bearer token that outlives the session, lands in browser history, and leaks through the `Referer` header; signing also requires an IAM permission the hosting platform's default credentials may not hold — a failure that appears only in production.

A virus-scan state field and gate exist on every uploaded file, with nothing scanning yet. It is present now because adding it later would mean retrofitting a state machine onto a collection that already has documents and deciding, at that point, what an unscanned legacy file counts as.

5.5 Application hardening

Control	State
Content Security Policy	<code>default-src 'self'</code> with explicit allowlists; nonce for inline structured data
Rate limiting	Applied to all public capture endpoints
Cron authentication	Shared secret compared in constant time
Structured data	Escaped before injection into the page

Control	State
Server action errors	Returned as typed results, not thrown — a thrown error in production becomes an opaque digest and blanks the screen mid-task
Dependency posture	Patched; npm audit fix --force is not run, as it downgrades the Admin SDK. Twelve transitive advisories with no clean fix are tracked, not chased.

One coupling is recorded explicitly: rate limiting is per-instance and in memory, which is correct *only* because the hosting configuration caps the backend at one instance. Raising the instance cap for traffic would silently multiply every limit. Moving limits into the database is a prerequisite for scaling out, and is noted in the configuration file itself so the coupling cannot be missed.

5.6 Audit log

Every consequential action writes an append-only entry — actor, dotted action name, entity, timestamp — on the same database batch as the action itself. There is no update path and no delete path in the module. Retention is two years, enforced by a database TTL policy rather than a scheduled cleanup, because a cron job that silently stops is how audit logs quietly become unbounded.

Three actions record after the fact rather than on the batch, and only these: deleting a post, deleting a job opening (the document is gone, so there is nothing left to batch against) and the bulk CSV import (whose writes are already chunked across several batches). Each is commented at the point it happens.

5.7 DPDP Act 2023

The site collects children's data — name, date of birth, age band. The Act's child provisions are the strictest part of it.

Requirement	Implementation
Verifiable parental consent	Consent captured at the point of collection
No behavioural tracking or targeted advertising directed at children	Analytics is consent-gated and does not load until consent is given
Named grievance officer	Field exists; the school must supply the name — the server logs a compliance warning at every start until it is filled in
Stated retention period	Published; CVs are deleted automatically at 365 days by a storage lifecycle rule, which matches what the policy promises
Processors named	Google (Firebase) and Resend named in the published policy

Requirement	Implementation
Data residency	Database migrated to Mumbai on 5 August 2026

Two open compliance items, both listed as actions in Section 11:

1. The **grievance officer name** is not yet supplied. Only the school can answer this.
2. The **published privacy policy still states that data is stored in the United States**. That was true until 5 August 2026 and is now stale. The database was migrated to Mumbai; the policy text must be corrected to match. Until it is, the school's public statement about where children's data lives is inaccurate.

6. Cost Model

6.1 The target, restated

The stated budget is under ₹1,000/month. That is not the real constraint — the free tier is:

Resource	Free allowance
Database document reads	50,000 / day
Database document writes	20,000 / day
Database storage	1 GiB
File storage	5 GiB
Application hosting	2M requests, 180k vCPU-seconds / month
Authentication	50,000 monthly active users

So the engineering goal is ₹0, not ₹1,000, leaving the entire ₹1,000 as headroom for an unusual month.

6.2 Why the naive design failed the target

Before the cost work, every admin page load fetched every lead of a type with no limit, each lead carrying its full note history:

Leads in database	Reads per dashboard load	At 200 loads/day
200	200	40,000 — 80% of the free tier
2,000	2,000	400,000 — 8x over
10,000	10,000	2,000,000 — 40x over

At 10,000 leads that is roughly ₹2,900/month from the dashboard alone, before the daily job's full scan. The target missed by three times, on a linear curve with no ceiling.

6.3 Post-implementation budget

Surface	Reads per action	Actions/day	Reads/day
Public pages (static / prerendered)	0	5,000 visits	0
Careers (regenerated hourly)	~5 per regeneration	24	120
Admin inbox (25-row page + counts)	26	200	5,200
Lead detail (document + first notes page)	11	100	1,100
Insights	1	96	96
Daily follow-up digest (bounded query)	~10	1	10
Total			≈ 6,500 / day

13% of the free read allowance. Writes land near 100/day against a 20,000 allowance. Room for roughly eight times the current volume before a single rupee is billed.

Crucially the curve is now **flat in record count** — 500 leads and 50,000 leads cost the same per dashboard load, because pages are capped and counts come from aggregation queries rather than from fetching documents in order to count them.

6.4 Third-party running costs

Service	Present cost	Scales with
Firebase (database, storage, hosting, auth)	₹0 within free tier	Usage beyond the allowances above
Domain <code>alfitrahsarjapura.in</code>	~₹1,200 / year	Fixed
Resend (transactional email)	₹0 within free tier	Volume beyond the free allowance
WhatsApp Cloud API (<i>when enabled</i>)	Per-conversation, billed by Meta	Number of conversations
SMS (<i>if enabled</i>)	Per message	Volume — the reason email is the default
Payment gateway (<i>when enabled</i>)	~2% + GST per transaction	Collections

6.5 Where the infrastructure is going

₹0/month is today's number, not a permanent guarantee. It holds because current volumes sit inside Google's free tier, and the design keeps them there. It is a starting position, not the long-term platform.

The intended direction, once funding is in place, is a move off the Firebase free tier onto **Cloudflare and AWS** — Cloudflare for DNS, CDN, edge caching and WAF, AWS for compute, database and object storage. The reasons are operational rather than financial:

- **Portability.** Firestore's data model and security rules are the one genuinely Google-specific part of this system. Everything else — the application, the server actions, the storage layer — is standard and moves without a rewrite.
- **Control.** Backend region, network policy, WAF rules and rate limiting at the edge are all fixed or absent on the current stack. On Cloudflare and AWS they are configuration.
- **Headroom.** Free-tier ceilings become a real constraint the moment the school runs multiple branches or opens the portal to every parent at once.

Indicative running cost after that move: **₹1,000–3,000/month** at current volumes, depending on which managed services are chosen. That is a real bill replacing a free one, and it buys control, not features.

The migration is **not priced in this proposal**. It is scoped and quoted separately when funding is confirmed, because the sensible shape of it depends on where the school is at that point.

7. Module Designs

Each module below states what is built and what remains. Effort for the remaining work is estimated in Section 8.

7.1 Public website & content management — *built*

Eleven pages covering about, programmes, admissions, campus life, news, syllabus, careers, parent resources, FAQ, contact and privacy. Search-optimised with structured data, branch-qualified so the Sarjapura branch ranks for its own name rather than competing with other franchises for the shared one.

News and events are editable by the school without a developer. Post bodies are plain text rendered as paragraphs and never as markup — see Section 2.4.

Remaining: gallery and faculty content types (the same machinery extends to them), per-page SEO copy.

7.2 Admissions CRM — *built*

Multiple capture surfaces, all landing in one place: the full enquiry form, a low-friction name-and-phone waitlist, a prospectus download, a site-wide WhatsApp call-to-action that encodes the page it was pressed on, and manual walk-in logging. Campaign attribution and referral source are captured on every lead.

Pipeline stages are configurable. The console provides a paginated inbox, a needs-attention triage view (overdue follow-ups and new leads untouched beyond 48 hours), inline stage changes, one-click WhatsApp follow-up from a template, follow-up dates with snooze, an activity timeline, tags, assignment, duplicate detection, bulk actions and CSV export. A daily reminder digest runs on a schedule.

The inbox is optimistic by design. Stage changes and snoozes apply immediately in the browser and revert if the server rejects them. Live database listeners are impossible here by design — client access to leads is denied — and would bill reads continuously for every open tab in any case. This took a stage change from a full collection re-read to one read and one write.

7.3 Teacher recruitment — *partially built*

Job openings, public applications with CV upload, a separate recruitment pipeline, and CV access restricted to authorised staff and streamed rather than linked.

Remaining: interview scheduling, candidate ratings, internal-only comments, portfolio links.

7.4 Student records — *built*

Own collection, transactional admission numbers, guardian records, status lifecycle, provenance link back to the originating enquiry, and bulk CSV import.

7.5 Attendance — *partially built*

Daily register per class, idempotent marking, monthly percentage, configurable statuses and non-school days, and a configurable low-attendance threshold.

Remaining: staff attendance (depends on the staff module), bulk marking across classes, CSV export, and trend analytics.

7.6 Fees — *partially built*

Immutable ledger, receipt numbering, derived balance with reconciliation, configurable payment methods, amounts stored in paise (never floating point).

Remaining: online collection, automated invoices and reminders, defaulter reporting.

7.7 Parent portal — *partially built*

Email-link and phone sign-in, server-resolved access to that parent's children only, read-only fee position and attendance, and access to resources shared with parents.

Remaining: two-way messaging, per-child document downloads, report card access (depends on the academics module).

7.8 Resource centre — *built*

Uploads with title, description, category, class section and academic year. Audience determines the storage location, and changing it moves the file. Three audiences exist because three resolve to a real sign-in: public, parents and staff. Students and teachers are deliberately absent — an audience nobody can sign in as is a checkbox that silently shares a file with no one. They arrive with the student-login and staff modules.

7.9 Notifications — *built*

Event-in, channel-out engine described in Section 3.5.2. Email and in-app channels configured; WhatsApp and SMS declared and awaiting provider credentials. Message templates are editable in the admin panel.

7.10 Configuration — *built*

Owner-only settings screen covering school identity, both pipelines, all taxonomies, academic year, attendance rules, notification events and templates, and feature flags.

7.11 Dashboards & analytics — *remaining*

One analytics view exists (sources, cumulative funnel, referrer leaderboard). The brief requests six role-specific dashboards.

Built last, deliberately. They are the most visible item and the least useful to build early: a principal's dashboard over a CRM with no task tracking shows confident numbers about nothing. Every dashboard reads from modules that must exist first.

7.12 Staff / HR — *remaining*

Teachers currently exist only as hired applications, not as employees. This module unblocks three things at once: staff attendance, the teacher resource audience, and role assignment managed by the school rather than by configuration.

7.13 Academics — *remaining*

Assessments, marks entry, and report card generation. The largest remaining module and the one with the most school-specific rules; requires the school's assessment scheme before design can be finalised.

7.14 Timetable — *remaining*

Class and teacher timetables with clash detection. Depends on the staff module.

7.15 Library & transport — *remaining, optional*

Listed in the brief; recommended for a later phase once the core operating modules are in daily use.

8. Delivery Sequence

8.1 Sequencing principle

Ordered by what unblocks the most downstream work, not by visibility. Configuration, the audit log and the notification engine were built first for exactly this reason — every later module sits on them. Dashboards are built last for the same reason, inverted.

8.2 Ground rules

- One phase = one pull request = one reviewable unit. No mixing concerns.
- Build, lint and tests green before a phase is opened for review.
- Every phase adds or updates at least one test covering what it changed.
- Data migrations ship with a dry-run mode and are idempotent.
- No new runtime dependency without an explicit call-out and written sign-off.
- Behaviour-preserving unless the phase explicitly changes behaviour.

8.3 Remaining phases

Effort is stated in developer-days. Elapsed time assumes a single developer and excludes the school's review time.

Core sequence — the platform the school actually runs on day to day.

Phase	Scope	Effort (days)	Unblocks
P1	Compliance close-out — grievance officer, privacy policy residency correction, backup workflow secret	1	Legal accuracy
P2	Staff / HR module — employee records, teacher identity, school-managed roles	12	P3, P4, teacher resources
P3	Attendance depth — staff attendance, bulk marking, CSV export, analytics	8	P6
P4	Recruitment depth — interview scheduling, ratings, internal comments, portfolio links	6	—

Phase	Scope	Effort (days)	Unblocks
P5	Online fee collection — payment gateway, invoices, automated reminders, defaulter report	10	—
P6	Dashboards — principal, admissions, CRM, attendance, HR, system	14	—
P7	Analytics depth — trends, campaign performance, scheduled monthly report	6	—
P12	Hardening, load verification, documentation, staff training, handover	8	—
	Core total	65	

Add-ons — genuinely useful, genuinely optional. Nothing in the core sequence depends on any of them, and each can be commissioned later, in any order, or never.

Phase	Scope	Effort (days)	Depends on
P8	Communication — WhatsApp and SMS channel adapters, broadcast, two-way parent messaging	12	—
P9	Academics — assessments, marks entry, report cards, portal access	18	—
P10	Timetable — class and teacher timetables, clash detection	10	P9 for room/teacher load, if P9 is taken
P11	Library & transport	14	—
	Add-on total	54	
	Everything (P1–P12)	119	

8.4 Indicative schedule

At four working days per week on this engagement, the core sequence spans approximately **16 weeks**; all four add-ons on top would take it to roughly **30 weeks**. The phases are independently deliverable and independently billable — the school may stop, reorder or defer at any phase boundary without stranded work.

Recommended order if the core is not commissioned at once: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P5 \rightarrow P6$. That closes the compliance gap, completes daily operations, gets money moving through the system, and only then builds the reporting layer over data that is finally complete.

9. Commercials

The core build is quoted at a flat **₹40,000 in total**. The per-phase figures below are that total divided across the core phases in proportion to the effort in Section 8.3, so the school can commission phases individually without the arithmetic changing. The four add-ons are priced separately and are not part of that total.

9.1 Basis

Item	Value
Engagement model	Fixed price per phase, apportioned from a flat total
Core total (P1–P7, P12)	₹40,000 for 65 developer-days
Add-ons (P8–P11)	Priced individually, commissioned only if wanted
Billing entity	DVLN Solutions LLP
Currency	INR, exclusive of GST where applicable
Quotation validity	30 days from issue

This is not a market rate. The same 119 developer-days quoted commercially at a blended ₹6,000/day would be ₹7,14,000; the figure here reflects the relationship with the school. The effort is stated in days regardless, so the school can see the size of what it is commissioning rather than only the price.

Fixed-price-per-phase is proposed rather than time and materials so the school carries no estimation risk within a phase, and rather than one lump sum so the school can stop at any boundary.

9.2 Core pricing

Phase	Scope	Days	Fee
P1	Compliance close-out	1	₹500
P2	Staff / HR module	12	₹7,500
P3	Attendance depth	8	₹5,000
P4	Recruitment depth	6	₹3,500

Phase	Scope	Days	Fee
P5	Online fee collection	10	₹6,000
P6	Dashboards	14	₹8,500
P7	Analytics depth	6	₹4,000
P12	Hardening, training, handover	8	₹5,000
	Core total	65	₹40,000

Recommended first commitment — P1, P2, P3, P5 and P6 (45 days, ₹27,500) delivers a complete daily-operations system with money moving through it and reporting over it. P7 and P12 close the core out.

9.2a Add-on pricing

Priced below the core's own rate, because none of this is needed for the school to run on the platform. Each is a standalone commission with its own start and acceptance.

Phase	Scope	Days	Fee
P8	Communication (WhatsApp / SMS / messaging)	12	₹6,000
P9	Academics	18	₹8,000
P10	Timetable	10	₹5,000
P11	Library & transport	14	₹4,000
	All four add-ons	54	₹23,000
	Everything (P1–P12)	119	₹63,000

9.3 Payment schedule

Per phase: 40% on phase start, 60% on acceptance. Invoices raised by DVLN Solutions LLP, payable within 15 days.

9.4 Support and maintenance — optional

Option	Fee	Includes
Warranty	Included, 30 days per phase	Defect correction against the accepted scope, at no charge
Support retainer	₹1,500 / month	Up to 8 hours of changes, dependency and security updates, backup and cost monitoring, response within one business day
Ad-hoc changes	₹1,000 / day	Billed against actual effort, no minimum, no retainer required

9.5 Running costs borne by the school

As detailed in Section 6.4: domain renewal (~₹1,200/year), and per-usage charges for WhatsApp, SMS or payment gateway only if and when those are enabled. Infrastructure is ₹0/month at present volumes on the current stack, and an indicative ₹1,000–3,000/month after the Cloudflare and AWS migration described in Section 6.5 — a migration that is scoped and quoted separately, when funding is confirmed.

9.6 Intellectual property

On settlement of all invoices for a phase, the source code and all deliverables of that phase belong to AI Fitrah Pre School, Sarjapura. The school holds its own accounts for every third-party service — Google Cloud, the domain registrar, the email provider — so there is no vendor lock-in and no dependency on the developer for continued operation.

10. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation
R1	Cost invariant broken by a future change, causing bill growth	Medium	High	Automated invariant tests reject the change; ₹500/month budget alert at 50/90/100%
R2	Data loss	Low	Critical	Point-in-time recovery enabled (7 days); weekly off-site export to a separate bucket with 90-day retention
R3	Parent sees another family's records	Low	Critical	Child access resolved server-side from the verified session only; never from client input. Highest-priority test coverage
R4	Compliance action over child data	Low	High	DPDP controls implemented (Section 5.7); two open items must close — grievance officer name and the stale residency statement
R5	Instance cap raised for traffic, silently multiplying rate limits	Medium	Medium	Coupling documented at the configuration site; database-backed limits are a stated prerequisite for scaling out
R6	Academics module rules differ materially from assumptions	Medium	Medium	P9 begins with a requirements workshop; effort re-confirmed before the phase is priced and started
R7	Free-tier limits exceeded as the school grows	Low	Medium	Current usage ~13% of the read allowance; overage is metered, not a cliff — ₹1,000/month buys roughly 20 million additional reads
R8	Key-person dependency on a single developer	Medium	Medium	Architecture, operations and design decisions documented in the repository; school owns all service accounts and all code
R9	Third-party pricing change (WhatsApp, gateway)	Medium	Low	Channels are adapters behind one interface; a provider swap replaces one function

#	Risk	Likelihood	Impact	Mitigation
R10	Scope grows within a phase	Medium	Medium	Phase scope fixed at start; additions quoted as a change request rather than absorbed

11. Assumptions, Dependencies & Acceptance

11.1 Assumptions

1. The school provides content — text, photographs, policy documents — within a reasonable period of request.
2. One named person at the school is available for review and sign-off at each phase boundary.
3. The school retains ownership of all third-party accounts and settles their charges directly.
4. Volumes remain within an order of magnitude of the projections in Section 6.
5. Staff have a device with a modern browser and an internet connection.
6. Modules not listed in Section 8.3 are out of scope for this proposal.

11.2 Actions required from the school

#	Action	Blocks	Owner
A1	Supply the grievance officer's name and email	DPDP compliance	School
A2	Approve the corrected data-residency statement in the privacy policy	Accuracy of a public legal statement	School
A3	Confirm the phase priority order in Section 8.4	Phase P2 start	School
A4	Confirm the commercial model in Section 9	All phases	School
A5	Provide the assessment and grading scheme	Phase P9 (academics)	School
A6	Nominate the staff who hold owner rights	Role split	School

11.3 Acceptance criteria

A phase is accepted when:

1. Every item in that phase's stated scope is demonstrable in the production environment.
2. Build, lint, unit tests and end-to-end tests pass on the main branch.
3. The cost invariant tests pass.
4. No defect of severity *critical* or *high* remains open against that phase.
5. Documentation affected by the phase is updated in the same change.

6. The school's nominated reviewer signs off in writing.

The 30-day warranty in Section 9.4 begins at acceptance.

Appendix A — Environment & Secrets

Configuration held outside the codebase. No secret is ever committed.

Name	Purpose	Required
ADMIN_EMAILS	Addresses permitted to sign in to the console	Yes
ADMIN_OWNERS	Subset permitted to delete records	Recommended
RESEND_API_KEY	Transactional email	Yes
INQUIRY_ADMIN_EMAIL	Recipient of enquiry alerts	Yes
INQUIRY_FROM_EMAIL	Sender address	Yes
CRON_SECRET	Authenticates the scheduled digest; must match in both the hosting secret and the CI secret	Yes
GCP_SA_KEY	CI credential for the weekly export	Open — see Appendix B
NEXT_PUBLIC_COMING_SOON	Gates the public site to a holding page	Optional
NEXT_PUBLIC_PROSPECTUS_URL	Prospectus download target	Optional

Appendix B — Operational Runbook Status

Verified 5 August 2026.

Item	State
Point-in-time recovery	Enabled, 7-day retention
Composite indexes	12 deployed, all ready
Security rules	Deployed — leads, students, attendance, payments all denied to clients
CV retention lifecycle	Deleted at 365 days, matching the published policy
Backup bucket	Created in Mumbai, 90-day lifecycle
First database export	Completed
Audit log retention	2-year TTL policy enforced by the database
Error alerting	Log metric and alert policy to the owner's email
Budget alert	₹500/month, alerting at 50 / 90 / 100%
Local service-account key file	Deleted; local development uses user credentials
Data residency	Migrated to Mumbai, verified by a full pre- and post-migration diff
Weekly backup job	Committed but failing until the CI credential is added — deliberately, because a backup you believe in but do not have is worse than none
Grievance officer	Open — school action A1
Privacy policy residency text	Open — school action A2

Appendix C — Glossary

Term	Meaning
Aggregation query	A database query returning a count or sum without transferring the underlying documents. Billed at a fraction of the cost of fetching them.
Append-only	A collection with no update and no delete path. Corrections are new entries.
DPDP Act 2023	India's Digital Personal Data Protection Act, whose child-data provisions govern this platform.
Free tier	Google's permanent no-cost usage allowance, distinct from a trial.
ISR	Incremental Static Regeneration — pages served as static files and rebuilt on a schedule or on edit, so visitors never trigger a database read.
Idempotent	An operation that produces the same result whether performed once or repeatedly.
Lead	An enquiry or job application, before it becomes a student or an employee.
Paise	One hundredth of a rupee. All money is stored as whole paise; floating-point currency is never used.
Server component	Page code executing only on the server, so credentials and queries never reach the browser.
TTL policy	Time-to-live — automatic deletion of expired records by the database itself, rather than by a scheduled job that can fail silently.

Prepared by DVLN Solutions LLP · 7 August 2026 · Confidential